

Identidad y gestión de accesos

De caja negra a herramienta que usas de verdad

Tu rol en el Learner Lab está preasignado y no puedes crear usuarios ni roles nuevos. Hoy no cambia esa regla — pero dejas de tratar IAM como algo que «ya viene configurado».

Aplicar un rol existente

Para que una instancia acceda a otros servicios sin credenciales

Detectar políticas mal escritas

Antes de que causen un problema real

Usuarios, roles, políticas y credenciales temporales

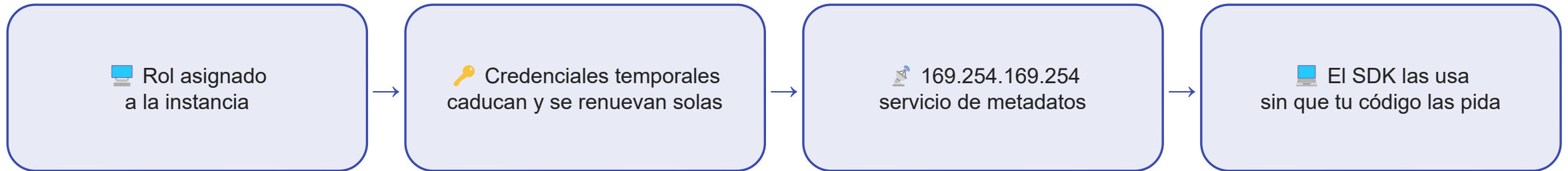
Ya llevas semanas usando una de estas piezas sin nombrarla

Pieza	Qué es	En el Learner Lab
Usuario	Identidad fija, con credenciales de larga duración	No puedes crear ninguno — no lo necesitas
Rol	Identidad que se presta temporalmente a quien la asume	Tienes uno preasignado (assumed-role)
Política	Documento que dice qué acciones están permitidas o denegadas	Puedes leerlas y corregirlas, no crear roles nuevos
Credenciales temporales	Claves de acceso con caducidad, al asumir un rol	Es lo que usa tu sesión ahora mismo

Toda llamada a la API va firmada con una credencial — la pregunta es de dónde sale, y ahí es donde un rol resuelve algo que un usuario no puede.

Por qué un rol resuelve lo que un usuario no puede

Si una instancia necesitara las claves fijas de un usuario, esas claves tendrían que vivir guardadas dentro de la instancia — el mismo problema de «credenciales en el código» que ya evitaste en el Tema 3.



Tu código nunca necesita saber dónde están esas claves ni pedir las explícitamente.

Por qué S3Storage no tiene ninguna clave de acceso

```
S3Client.builder().build()
```

En la Actividad 5.2, la clase que guarda las fotos de Escaparate en S3 crea su cliente así, sin pasarle ninguna credencial. El SDK pregunta solo al servicio de metadatos qué credenciales temporales tiene la instancia ahora mismo, y las usa.

Si la instancia cambia, nada se rompe

Si esa instancia se termina y otra la sustituye, la nueva pregunta lo mismo y recibe sus propias credenciales — nadie ha copiado ni rotado ninguna clave a mano.

Anatomía de una política

El mismo formato JSON de la sesión 1 — con una estructura fija

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::escaparate-imagenes-<id>/*",
  "Condition": { "Bool": { "aws:SecureTransport": "true" } }
}
```

Campo	Responde a	En el ejemplo
Effect	¿Permite o deniega?	Allow
Action	¿Qué operación?	GetObject y PutObject (leer y escribir)
Resource	¿Sobre qué recurso concreto?	Objetos del bucket de imágenes
Condition	¿Bajo qué circunstancia adicional?	Solo si la petición viaja cifrada (HTTPS)

Leer una política, y comprobarlo con el simulador

«Permite leer y escribir objetos del bucket de imágenes, pero solo si la conexión va cifrada» — cuatro campos, una frase.

Simulador de políticas de IAM

Eliges una política, una acción y un recurso concretos, y te responde «permitido» o «denegado» — sin ejecutar nada de verdad contra tu cuenta.

Con esta política: s3:GetObject sobre un objeto del bucket → Permitido. s3:DeleteObject sobre ese mismo objeto → Denegado, porque esta política nunca menciona esa acción.

Cómo se evalúa una política: quién gana

El resultado nunca se decide sumando políticas — siempre con este orden fijo



Un Deny explícito gana siempre, aunque otra política diga Allow. Si ninguna política menciona una acción, queda denegada por defecto, nunca permitida «por si acaso».

Principio de mínimo privilegio y MFA

 Permisos amplios
«por si acaso»



 Un fallo tiene
alcance mucho mayor

 Mínimo privilegio



 Un fallo queda
contenido

Un comodín de más es el error más común

Una política con "Action": "s3:*" en vez de "s3:GetObject" da permiso para borrar y cambiar todo el almacenamiento, cuando quizás solo hacía falta leer.

MFA: una segunda prueba de identidad

Un código temporal, además de la contraseña, para operaciones sensibles. En el Learner Lab no lo configuras tú, pero en una cuenta real protege contra una contraseña filtrada.

Gestión de secretos: nunca en el código ni el repositorio

Ya aplicaste esta regla en el Tema 3 con la contraseña de la base de datos — hoy se generaliza a cualquier secreto: claves de API, tokens, certificados.

 Secreto

malo →

 Escrito en el código
o fichero versionado

 Secreto

bueno →

 Secrets Manager
resuelto en tiempo de ejecución

Un secreto filtrado sigue expuesto aunque lo borres después

El historial de Git conserva versiones anteriores — borrarlo en el commit siguiente no lo elimina. La única solución correcta es rotarlo, nunca solo quitarlo del código.

Registro de auditoría: quién, cuándo y desde dónde

CloudTrail guarda un histórico de quién ha hecho qué, cuándo y desde dónde — cada llamada a la API queda registrada, por consola o por CLI.

Auditoría aplicada a un caso real

Un grupo de seguridad tenía una regla que no debería estar ahí. Las métricas y los registros te dijeron qué estaba fallando; el registro de auditoría te dice quién cambió esa regla y cuándo — la pieza que faltaba para entender el origen, no solo el síntoma.

Ideas clave

1

Un rol se presta temporalmente, con credenciales que caducan; un usuario tiene credenciales fijas — el Learner Lab usa un rol preasignado.

2

Una política se lee en cuatro campos: efecto, acción, recurso y condición.

3

Gana siempre el orden fijo: un Deny explícito primero, luego un Allow explícito, y si nadie dice nada, deniega por defecto.

4

El mínimo privilegio limita el alcance de cualquier fallo; un comodín de más es el error más común que lo rompe.

5

Ningún secreto va nunca en texto plano en un fichero versionado — si uno se filtra, la solución es rotarlo, no solo borrarlo.

6

El registro de auditoría responde a quién ha hecho qué y cuándo — la pieza que completa el diagnóstico, más allá de métricas y registros.